CS543 WEB SERVICES — ASSIGNMENT 4

Smart Contact Manager (SCM)

HTTP REST resources and headers — Java Spring Boot implementation

Team ID	12
Team members	Aman Bali — 20251651015 Bhupesh Kumar — 20251651032
Technology	Java 21 · Spring Boot 3.2.5 · Maven · JUnit 5 · MockMvc
Service port	8090 (existing SCM Contact Service: 8082)

This report accompanies the source folder, OpenAPI contract, tests and NOTES.md. The Java implementation extends the SCM project; no Python/Flask service is used.

CS543 Assignment 4 — REST resources in Smart Contact Manager (SCM)

Team ID: 12

Aman Bali — 20251651015

Bhupesh Kumar — 20251651032

Technology stack and project fit

This submission uses **Java 21, Spring Boot 3.2.5, Spring Web, Jakarta Bean Validation, Maven, JUnit 5, Mockito and MockMvc**. It is not a Python/Flask implementation. The new optional `contact-share-service` runs on port `8090` and calls the existing SCM Contact Service on port `8082`. It owns share-request records only; it does not duplicate or take ownership of the contact records.

Existing SCM ports documented in the uploaded repository:

Service	Port	Responsibility
API Gateway (Node/Express)	8080	Public gateway and API routing
Auth Service (Spring Boot)	8081	Authentication and roles
Contact Service (Spring Boot)	8082	Contact CRUD and search
Media Service	8083	Contact photo/media
Notification Service	8084	Email notification
Audit Service	8085	Audit/activity records
Integration Service	8086	SOAP/WSDL integration
Assignment 4 contact-share extension	8090	Share-request resource lifecycle

A1–A4 — Resource operations

Method	URL	What it does	Success	Failure
POST	`/v1/contact-shares`	Creates a share request for an existing SCM contact; requires `Idempotency-Key`	`201 Created` + `Location`	400 malformed/invalid body or missing key; 409 key reused with a different payload; 422 referenced contact missing; 502/503 downstream error
GET	`/v1/contact-shares/{shareId}`	Reads one share request	`200 OK`	404 unknown ID
GET	`/v1/contact-shares?status=pending`	Lists requests, optionally filtered by status	`200 OK`	422 unsupported status
POST	`/v1/contact-shares/{shareId}/cancellation`	Cancels a pending request using a sub-resource	`200 OK`	404 unknown ID; 409 request already cancelled or not pending

The resource nouns are `contact-shares` and `cancellation`. I avoided RPC-style paths such as `/cancelContactShare`.

A5 — Hard resource-design choice

Cancellation is a state transition rather than a plain update or delete. It is represented as `POST /v1/contact-shares/{shareId}/cancellation`, which makes the cancellation action a sub-resource and keeps verbs out of the main resource URL. I rejected `/cancelContactShare` because it would expose an RPC-style endpoint and make the URL less resource-oriented. A second cancellation returns `409 Conflict`, so a client can tell that the requested state transition is no longer valid.

C1–C8 — Implementation details

• `model/ContactShare.java` stores the record; `ContactShareResponse.java` is the public representation. The internal idempotency key is not serialized into the response.

• `store/ShareStore.java` owns the in-memory store and idempotency-key map. No other SCM service imports this store.

• `error/GlobalExceptionHandler.java` returns a consistent JSON problem shape: `type`, `title`, `status`, `detail`.

• `openapi.yaml` documents the API contract and response cases.

• `ContactServiceClient.java` makes a real HTTP GET to the configured Contact Service URL. The URL comes from `CONTACT_SERVICE_URL`, with a connect/read timeout and bounded exponential backoff plus jitter. 4xx responses are not retried.

• The store is in-memory for the assignment. Records are lost on restart; a production service should use persistent storage.

• Create is safely retryable when the same `Idempotency-Key` and same normalized request body are reused. Reusing the key for a different body returns `409 Conflict`.

D3 — Dependency fallback

If the Contact Service cannot confirm that the contact exists, creation fails with `503 Service Unavailable` (or `502 Bad Gateway` for a bad downstream response). The service does not create a share request without a verified contact, because that would leave a dangling reference and mislead the user. A referenced contact that is confirmed missing returns `422 Unprocessable Entity`.

Answers required by the assignment

1. WSDL and OpenAPI line counts

Count the files in this submission and the SCM-aligned Assignment 3 reference with:

```
wc -l references/scm-telecom-directory.wsdl openapi.yaml
```

2. SOAP fault to HTTP problem

The SCM-aligned Assignment 3 illustrative fault uses `soap:Server`, `Telecom directory temporarily unavailable`, and detail code `UPSTREAM_UNAVAILABLE`. At the REST boundary, a transient dependency failure becomes `503 Service Unavailable` with a JSON body such as:

```
{
  "type": "about:blank",
  "title": "Service Unavailable",
  "status": 503,
  "detail": "SCM Contact Service is temporarily unavailable."
}
```

Returning the failure inside `200 OK` is misleading: clients, gateways, caches and monitoring may classify the request as successful and fail to apply normal retry/error handling. The XML fault in Assignment 3 was an illustrative downstream example; it is not a named fault declared by the current SCM WSDL.

3. UDDI publish/find/bind

The concepts remain, but this assignment does not run a UDDI registry. **Publish** is committing the OpenAPI contract and service address; **find** is locating the contract and configured service; **bind** is using the base URL and HTTP path to call the operation. The OpenAPI document and environment-based configuration take over most of this practical work.

4. Where schema validation occurs

Jakarta Bean Validation annotations on `CreateShareRequest` and `@Valid` on the controller method enforce required fields, email format and message length. The OpenAPI schema documents the rules but does not automatically enforce them at runtime. Without validation, an empty contact reference or invalid email could reach business logic or the downstream Contact

Service.

5. Where SOAP remains preferable

SCM's existing telecom-directory integration remains a good SOAP boundary when an enterprise provider requires a WSDL-defined XML contract. The guarantee being purchased is a shared, machine-readable contract for operation names, message structure and SOAP binding that both sides can validate and generate clients from. SOAP itself does not guarantee delivery or authorization; those require separate mechanisms.

Run and verify

```
mvn clean test
mvn spring-boot:run
```

Configure the Contact Service endpoint if needed:

```
export CONTACT_SERVICE_URL="http://localhost:8082/v1/contacts"
```

The current `curl-transcript.txt` is a template, not live evidence. Capture actual responses after both services are running. Do not report tests or OpenAPI validation as passed until the commands have run successfully in your environment.